

Line-by-Line Technical Analysis

Module Architecture & Implementation Details: `billing.py`

TARGET FILE <code>billing.py</code>		TOTAL LINES 207 Lines	LANGUAGE Python 3.x	DOMAIN Payment Processing
#	SOURCE CODE	TECHNICAL EXPLANATION		
1	<code>import os</code>	Imports the standard operating system interface module for file/path manipulation.		
2	<code>import json</code>	Imports the JSON encoding and decoding library for local storage persistence.		
3	<code>import re</code>	Imports the regular expression module for string validation patterns (UTR/RRN verification).		
4	<code>import time</code>	Imports time utilities for generating standard ISO-like human-readable timestamps.		
5	<code>import math</code>	Imports mathematical operations to verify numeric bounds (e.g., checking for NaN/Infinity).		
6	<code>from typing import Tuple, List, Dict, Any</code>	Imports explicit type hinting constructs for type safety and clarity across payment routines.		
7		Blank vertical spacer line separating imports from path definitions.		
8	<code># File Paths</code>	Section header comment designating system storage file path definitions.		
9	<code>DATA_DIR = os.path.abspath(...)</code>	Resolves the absolute path to the parent directory's data folder relative to this file.		
10	<code>PAYMENTS_FILE = os.path.join(...)</code>	Constructs the full absolute filepath for the local <code>payments.json</code> database file.		
11		Blank vertical spacer line separating paths from status constants.		
12	<code># Status Constants</code>	Section comment indicating state definitions used across payment lifecycles.		
13	<code>STATUS_PENDING = "PENDING"</code>	Constant for full-payment claims awaiting administrative manual verification.		
14	<code>STATUS_APPROVED = "APPROVED"</code>	Constant denoting a payment claim that has been successfully verified by an admin.		
15	<code>STATUS_PARTIAL = "PARTIAL"</code>	Constant indicating a transaction where paid amount is less than the required amount.		
16	<code>STATUS_TOPUP_DONE = "TOPUP_DONE"</code>	Constant set when top-up payments meet the deficit and await secondary approval.		
17	<code>STATUS_FLAGGED = "FLAGGED"</code>	Constant indicating a transaction flagged for invalid amounts, duplicates, or errors.		
18	<code>STATUS_INVALID_UTR = "INVALID_UTR"</code>	Constant signifying a UTR identifier failing strict formatting validation checks.		
19		Blank vertical spacer line separating constants from function implementations.		
20		Blank vertical spacer line.		
21	<code>def _ensure_data_dir():</code>	Private helper function declaration to verify/create target storage folders.		
22	<code>"""Ensures data dir..."""</code>	Docstring explaining folder creation purpose.		
23	<code>os.makedirs(DATA_DIR, exist_ok=True)</code>	Creates <code>DATA_DIR</code> recursively if missing; ignores existing path error.		

#	SOURCE CODE	TECHNICAL EXPLANATION
36		Blank vertical spacer line.
37		Blank vertical spacer line.
38	<code>def save_payments(payments: List...):</code>	Function declaration to serialize and persist payment records back to disk.
39	<code>"""Saves payment records..."""</code>	Docstring describing disk save operation.
40	<code>_ensure_data_dir()</code>	Guarantees destination folder existence prior to file write attempt.
41	<code>with open(PAYMENTS_FILE, "w"...) as f:</code>	Opens payment storage file in overwrite write mode with UTF-8 encoding.
42	<code>json.dump(payments, f, indent=2...)</code>	Formats dictionary list to JSON with 2-space indentation and non-ASCII support.
43		Blank vertical spacer line.
44		Blank vertical spacer line.
45	<code>def validate_utr_format(utr: str) -> bool:</code>	Utility function verifying whether a string matches Indian 12-digit UTR formatting.
46	<code>"""Validates Indian UPI..."""</code>	Docstring specifying strict 12-digit UPI/RRN validation constraints.
47	<code>cleaned_utr = utr.strip()</code>	Strips leading and trailing whitespace from the provided UTR input string.
48	<code>return bool(re.match(r"^\d{12}\$", cleaned_utr))</code>	Evaluates regex matching exactly 12 numeric digits; returns boolean status.
49		Blank vertical spacer line.
50		Blank vertical spacer line.
51	<code>def submit_payment(...) -> Tuple...:</code>	Core entrypoint function processing initial payment proof submissions.
52	<code>plan: str,</code>	Parameter: Name of plan purchased (e.g., Enterprise/Pro).
53	<code>required_amount: float,</code>	Parameter: Target amount required to fully satisfy the purchase.
54	<code>amount_paid: float,</code>	Parameter: Claimed monetary amount paid by user in transaction proof.
55	<code>evals: Any,</code>	Parameter: Associated evaluation parameters, tokens, or feature metadata.
56	<code>utr: str</code>	Parameter: Bank transaction UTR reference code string.
57	<code>) -> Tuple[str, str, float]:</code>	Specifies return signature: Tuple containing (Status, User Message, Deficit Balance).
58	<code>"""Submits and processes..."""</code>	Multi-step workflow summary docstring detailing pipeline logic.
59	<code>utr_clean = utr.strip()</code>	Cleans raw UTR parameter by removing boundary whitespaces.
60		Blank vertical spacer line.
61	<code># Step 1: Validate UTR format</code>	Phase indicator comment for standard UTR format verification step.
62	<code>if not validate_utr_format(utr_clean):</code>	Calls standard regex check; branches if UTR format is non-compliant.
63	<code>return (</code>	Returns immediate error tuple upon invalid UTR syntax detection.
64	<code>STATUS_INVALID_UTR,</code>	Sets return state status constant to STATUS_INVALID_UTR.
65	<code>"❌ Invalid UTR ID!...",</code>	Provides actionable user error detailing requirement of exactly 12 digits.
66	<code>required_amount</code>	Returns original required amount as remaining balance since payment rejected.
67	<code>)</code>	Closes error tuple construct.
68		Blank vertical spacer line.

#	SOURCE CODE	TECHNICAL EXPLANATION
76	<code>if (</code>	Opens compound conditional check for duplicate transaction matching.
77	<code>pmt.get("utr") == utr_clean</code>	Checks if current UTR matches primary UTR of existing record.
78	<code>or utr_clean in pmt.get(...)</code>	Checks if UTR was previously used in top-up submission arrays.
79	<code>):</code>	Closes conditional block.
80	<code>return STATUS_FLAGGED...</code>	Blocks duplicate submission, flags transaction, and returns zero remaining balance.
81		Blank vertical spacer line.
82	<code># Step 3: Calculate balance</code>	Phase indicator comment for deficit assessment step.
83	<code>remaining = max(0.0, required...)</code>	Computes deficit balance; caps lower bound at 0.0 to prevent negative remaining values.
84		Blank vertical spacer line.
85	<code>if amount_paid >= required_amount:</code>	Branches based on whether paid amount satisfies total required cost.
86	<code>status = STATUS_PENDING</code>	Sets state to PENDING, awaiting admin verification for full payment.
87	<code>msg = f"✅ Payment proof..."</code>	Formats success feedback string with formatted claim amount.
88	<code>else:</code>	Branches for underpayment/partial payment scenarios.
89	<code>status = STATUS_PARTIAL</code>	Sets record state to PARTIAL status requiring top-up or review.
90	<code>msg = f"⚠️ Partial payment..."</code>	Formats detailed warning string breaking down claimed paid vs. deficit amounts.
91		Blank vertical spacer line.
92	<code>new_record = {</code>	Instantiates dictionary structure representing the payment transaction record.
93	<code>"utr": utr_clean,</code>	Key: Cleansed primary 12-digit UTR string.
94	<code>"plan": plan,</code>	Key: Plan subscription name label.
95	<code>"required_amount": required_amount,</code>	Key: Total plan price expectation integer/float.
96	<code>"amount_paid": amount_paid,</code>	Key: Initial sum paid by customer.
97	<code>"remaining_balance": remaining,</code>	Key: Calculated remaining outstanding balance.
98	<code>"evals": evals,</code>	Key: Attached evaluation units/metadata.
99	<code>"status": status,</code>	Key: Transaction lifecycle status string.
100	<code>"timestamp": time.strftime(...),</code>	Key: ISO formatted submission time timestamp.
101	<code>"topup_utrs": []</code>	Key: Initialized empty array for tracking subsequent top-up reference UTRs.
102	<code>}</code>	Closes dictionary instantiation block.
103		Blank vertical spacer line.
104	<code>payments.append(new_record)</code>	Appends the new record dictionary to the in-memory payments list.
105	<code>save_payments(payments)</code>	Persists updated payments array to JSON file storage.
106	<code>return status, msg, remaining</code>	Returns submission outcome tuple containing status, message, and remaining balance.
107		Blank vertical spacer line.
108		Blank vertical spacer line.
109	<code>def submit_topup(...) -> Tuple...:</code>	Public handler routine processing top-up submissions to satisfy existing partial balance.
110	<code>"""Records a top-up claim..."""</code>	Docstring explaining top-up mapping and deficit reconciliation operations.

#	SOURCE CODE	TECHNICAL EXPLANATION
116	"❌ Invalid Top-Up...",	Error message detailing failed formatting requirement on top-up submission.
117	0.0	Returns 0.0 balance indicator for invalid attempt.
118	)	Closes error tuple construct.
119		Blank vertical spacer line.
120	if not isinstance(topup_amount...):	Validates topup_amount parameter numeric sanity and positivity.
121	return STATUS_FLAGGED...	Rejects invalid numeric top-up values; flags transaction attempt.
122		Blank vertical spacer line.
123	payments = load_payments()	Retrieves payment system database records from local storage.
124		Blank vertical spacer line.
125	# A UTR must be unique...	Comment emphasizing global uniqueness invariant across main UTRs and top-up UTRs.
126	if any(	Evaluates list comprehension checking if UTR is already recorded in system.
127	pmt.get("utr") == topup_utr_clean	Checks matching against primary transaction UTR keys.
128	or topup_utr_clean in...	Checks matching against existing secondary topup_utrs lists.
129	for pmt in payments	Loops through every existing payment dictionary in database.
130	):	Closes generator expression logic for duplicate detector.
131	return STATUS_FLAGGED...	Rejects top-up if proposed UTR identifier has already been used anywhere.
132		Blank vertical spacer line.
133	for pmt in payments:	Loops to find parent payment record matching parent_utr.
134	if pmt.get("utr") == parent_utr:	Matches target parent payment record by primary UTR key.
135	if pmt.get("status") not in...	Validates parent transaction status; ensures it is expecting top-up payments.
136	return STATUS_FLAGGED...	Blocks top-up attempt on non-partial/closed accounts; returns current remaining balance.
137	if pmt.get("remaining_balance"...) <= 0:	Checks if remaining monetary obligation has already been fulfilled.
138	return STATUS_FLAGGED...	Prevents unnecessary top-up when remaining balance is already 0.0.
139	if "topup_utrs" not in pmt:	Defensive check ensuring topup_utrs list exists on legacy records.
140	pmt["topup_utrs"] = []	Initializes missing array key safely if missing.
141		Blank vertical spacer line.
142	# Update totals	Comment block for state update and balance recalculation logic.
143	pmt["topup_utrs"].append(...)	Appends cleansed top-up UTR identifier to parent's tracking history array.
144	pmt["amount_paid"] += topup_amount	Increments cumulative amount paid sum on parent record.
145	new_remaining = max(0.0...)	Recalculates updated remaining deficit, bounded at a minimum of 0.0.
146	pmt["remaining_balance"] = ...	Updates top-level record's remaining balance value.
147		Blank vertical spacer line.
148	if new_remaining == 0.0:	Evaluates if top-up fully satisfied required payment total.
149	pmt["status"] = STATUS_TOPUP_DONE	Transitions status to TOPUP_DONE for admin review.
150	msg = f"✅ Top-up proof..."	Sets success feedback notification string.
151	else:	Handles ongoing partial payment scenario if balance remains > 0.

#	SOURCE CODE	TECHNICAL EXPLANATION
156	<code>return pmt["status"], msg...</code>	Returns tuple containing updated status, user message, and remaining balance.
157		Blank vertical spacer line.
158	<code>return STATUS_FLAGGED, "❌ Original..."</code>	Fallback return if parent UTR is not found in database records.
159		Blank vertical spacer line.
160		Blank vertical spacer line.
161	<code>def approve_payment(utr: str) -> bool:</code>	Administrative control action to mark payment record as fully verified and approved.
162	<code>"""Manually approves a payment..."""</code>	Docstring explaining admin authorization mutation.
163	<code>payments = load_payments()</code>	Fetches database payments array from file.
164	<code>for pmt in payments:</code>	Iterates through registered payment entries.
165	<code>if pmt.get("utr") == utr:</code>	Locates target transaction using primary UTR identifier.
166	<code>pmt["status"] = STATUS_APPROVED</code>	Sets state status to APPROVED.
167	<code>pmt["remaining_balance"] = 0.0</code>	Clears remaining balance to 0.0 upon manual admin authorization.
168	<code>save_payments(payments)</code>	Saves changes back to local JSON database file.
169	<code>return True</code>	Returns boolean success indicator flag.
170	<code>return False</code>	Returns False if target payment record was not found.
171		Blank vertical spacer line.
172		Blank vertical spacer line.
173	<code>def flag_partial(utr: str) -> bool:</code>	Admin action routine marking a payment submission manually as incomplete/partial.
174	<code>"""Flags a payment record..."""</code>	Docstring explaining manual partial status override routine.
175	<code>payments = load_payments()</code>	Loads current payment records array.
176	<code>for pmt in payments:</code>	Iterates over payment records.
177	<code>if pmt.get("utr") == utr:</code>	Matches transaction record by primary UTR string.
178	<code>pmt["status"] = STATUS_PARTIAL</code>	Mutates record status value to STATUS_PARTIAL.
179	<code>save_payments(payments)</code>	Persists state changes back to disk storage.
180	<code>return True</code>	Returns True upon successful record modification.
181	<code>return False</code>	Returns False if UTR lookup fails.
182		Blank vertical spacer line.
183		Blank vertical spacer line.
184	<code>def get_pending_payments() -> List[Dict[str, Any]]:</code>	Retrieves all payment transactions currently requiring administrative review.
185	<code>"""Returns all payment records that need admin attention."""</code>	Docstring explaining active queue query behavior.

#	SOURCE CODE	TECHNICAL EXPLANATION
186	<code>payments = load_payments()</code>	Loads full list of payment records from disk.
187	<code>return [p for p in payments if p.get("status") in (STATUS_PENDING, STATUS_PARTIAL, STATUS_TOPUP_DONE)]</code>	Filters and returns records matching PENDING, PARTIAL, or TOPUP_DONE statuses.
188		Blank vertical spacer line.
189		Blank vertical spacer line.
190	<code>def get_partial_by_utr(utr: str) -> Dict[str, Any]:</code>	Queries and fetches a single active partial/topup payment record by its UTR.
191	<code>"""Retrieves a specific partial payment record by UTR."""</code>	Docstring describing single transaction partial lookup method.
192	<code>payments = load_payments()</code>	Deserializes JSON payment database into memory.
193	<code>for p in payments:</code>	Loops through records array.
194	<code>if p.get("utr") == utr and p.get("status") in (STATUS_PARTIAL, STATUS_TOPUP_DONE):</code>	Checks matching UTR key and verifies status is either PARTIAL or TOPUP_DONE.
195	<code>return p</code>	Returns matched payment record dictionary.
196	<code>return {}</code>	Returns empty dictionary if no matching active partial record exists.
197		Blank vertical spacer line separating helper function.
198		Blank vertical spacer line.
199	<code># Utility exported helpers</code>	Section comment indicating exported helper routines for external integration.
200	<code>def clear_all_payments() -> bool:</code>	Utility function declared to purge stored transaction records.
201	<code>"""Clears all payments database for testing."""</code>	Docstring explaining testing/reset intent of clear operation.
202	<code>save_payments([])</code>	Overwrites database file with empty list structure.
203	<code>return True</code>	Returns boolean success status upon completion.
204		Blank vertical spacer line.
205		Blank vertical spacer line.
206	<code># End of billing.py script</code>	Closing comment marking final line of file module.
207		Final EOF newline character ending file execution.